

Modül 03: Akış Kontrolü

Java'da döngüler ve koşullar, daha önce kod yazmış biri için pek sürpriz içermez. Burada üç şey dikkatinizi hak ediyor: Java'da doğruluk kavramı yok, modern `switch` tanımış olabileceğinizden gerçekten daha iyi, ve etiketli break çoğu dilde bulunmayan bir sözdizimi.

Neler Öğreneceksiniz

- `if (name)` neden derlenmez ve bunun neden bir özellik olduğu
- Kısa devre değerlendirmenin bir optimizasyon değil doğruluk aracı olduğu
- Switch ifadeleri, çok etiketli durumlar ve `yield`
- Kapsayıcılık denetimi ve neden sıklıkla `default` dalı istemediğiniz
- Her döngü biçimi ve hangisine uzanacağınız
- İç içe döngüler için etiketli break

Başka Bir Dilden Geliyorsanız

	Python / JS	Java
Koşul	herhangi bir değer	yalnızca <code>boolean</code>
Boş string yanlış sayılır	evet	yanlış sayılan hiçbir şey yok
String üzerinde <code>switch</code>	JS evet, Python 3.10+ match	evet
Alt duruma düşme	JS evet	yalnızca eski ifade biçiminde
İç içe döngüden çıkma	bayrak değişkeni	<code>break</code> etiket;
<code>for x in xs</code>	yerleşik	<code>for (var x : xs)</code>

En büyük uyum, ilk satır. Java `if (items)` yazmanıza izin vermez ve boş bir listenin ne anlama geldiğine sizin adınıza karar vermez.

Ders

Doğruluk kavramı yok

```
String name = "";  
if (name) { } // does not compile  
if (0) { }    // does not compile
```

Karşılaştırmayı açıkça yazarsınız:

```
if (!name.isEmpty()) { ... }
```

Daha fazla yazım, karşılığında ezberlenecek yanlış değerler tablosu ve `"0"` ya da `[]` yanlış sayılır mı tartışması yok. Her koşul tam olarak neyi test ettiğini söyler.

Koruma olarak kısa devre

`&&` sol işlenen cevabı zaten belirlediğinde sağ işleneni atlar. Bu yalnızca bir performans ayrıntısı değil, null'a karşı korunma biçiminizdir:

```
if (maybe != null && maybe.length() > 3) { ... }
```

İkisini ters çevirin ve fırlatır. `&&` işlenenlerinin sırası bir doğruluk kararıdır.

`&` ve `|` de boolean üzerinde çalışır ve kısa devre **yapmaz**. Orada onları neredeyse hiç istemezsiniz. Asıl işleri bit düzeyinde:

```
6 & 3 = 2      6 | 3 = 7      6 ^ 3 = 5      6 << 1 = 12
```

Switch, iyi sürüm

Bir switch **ifadesi** değer üretir. [SwitchExpressions.java](#) dosyasına bakın:

```
String kind = switch (day) {  
    case 1, 2, 3, 4, 5 -> "weekday";  
    case 6, 7 -> "weekend";  
    default -> "not a day";  
};
```

`break` yok, dal başına birden fazla etiket var, ve bütünü atayabileceğiniz bir ifade. Bir dal birden fazla ifade gerektirdiğinde blok ve `yield` kullanın:

```
int workload = switch (day) {  
    case 6, 7 -> {  
        int base = 10;  
        yield base * 2;  
    }  
    default -> 8;  
};
```

`yield` var çünkü `return` tüm metottan çıkmaya çalışırdı.

Kapsayıcılık, asıl önemli kısım

Bu metodun `default` dalı yok ve derleyici kabul ediyor:

```
String describe(Light light) {  
    return switch (light) {  
        case RED -> "stop";  
        case AMBER -> "get ready";  
        case GREEN -> "go";  
    };  
}
```

Üç sabit, üç dal, eksiksiz. Şimdi `SwitchMustBeExhaustive.java` dosyasının yaptığı gibi `GREEN` dalını silin:

```
error: the switch expression does not cover all possible input values  
    return switch (light) {  
        ^  
missing patterns:  
    Light.GREEN
```

Derleyici unuttuğunuz durumu ismen söylüyor.

Bu, ilk bakışta tersmiş gibi duran bir tavsiyeye götürür:

Girdi kapalı bir kümeyseniz `default` dalını yazmayın.

`default` olmadan `Light` enum'una dördüncü bir sabit eklemek, güncellenmesi gereken her switch'te derlemeyi kırar ve hata her birini tek tek gösterir. `default` ile hepsi derlenmeye devam eder ve yeni sabiti sessizce yedeğe yönlendirir. Bugün aldığınız bir derleme hatası, üretimde bulacağınız bir hatadan iyidir.

Alt duruma düşme ve neden kaldırıldı

Eski ifade biçimi, bir şey durdurana kadar sonraki durumlara girerek çalışır.

`WhyFallThroughWentAway.java` `n = 2` ile ikisini de gösteriyor:

```
n = 2, old statement form:  
    two  
    three  
n = 2, arrow form:  
    two
```

Eski biçim 2 durumuyla eşleşti, yazdırdı, sonra hiçbir şey onu durdurmadığı için 3 durumuna düştü. Bir eksik anahtar kelime, ve yanlış davranış doğru gibi okunuyor.

Alt duruma düşme ara sıra durumları gruplamak için kullanılırdı. Ok biçimi bunu virgülle ve risk olmadan yapıyor: `case 1, 2, 3 -> handleSmall();`

Döngüler

Dört biçim, ve seçim genellikle apaçık. `Loops.java` dosyasından:

Biçim	Ne zaman
<code>for (var x : xs)</code>	indekse ihtiyacınız yoksa. Çoğu zaman.
<code>for (int i = 0; ...)</code>	indeks, birden farklı adım ya da geriye gitme gerekiyorsa
<code>while</code>	tekrar sayısı baştan bilinmiyorsa
<code>do-while</code>	gövde en az bir kez çalışmalıysa. Nadir.

Etiketli break, çoğu dilde karşılığı olmayan tek döngü sözdizimidir. İçteki bir döngüden dıştakinden çıkar:

```
search:
for (int row = 0; row < grid.length; row++) {
    for (int col = 0; col < grid[row].length; col++) {
        if (grid[row][col] == target) {
            break search; // leaves both loops
        }
    }
}
```

Onsuz bir bayrak değişkeni ve dış döngüde fazladan bir koşul gerekir, ki bu da çürüyen türden koddur. Ölçülü kullanın: bir metotta birden fazla etiket genellikle o metodun bölünmek istediği anlamına gelir.

Çalıştırın

```
java modules/03-control-flow/examples/OperatorsAndConditions.java
java modules/03-control-flow/examples/SwitchExpressions.java
java modules/03-control-flow/examples/WhyFallThroughWentAway.java
java modules/03-control-flow/examples/Loops.java

# Bilerek başarısız olur. Hangi durumu isimlendirdiğini okuyun.
java modules/03-control-flow/examples/SwitchMustBeExhaustive.java

./scripts/verify-examples.sh modules/03-control-flow
```

Sık Yapılan Hatalar

&& işlenenlerini ters sırada yazmak. `maybe.length() > 3 && maybe != null` null kontrolü hiç çalışmadan fırlatır.

Enum üzerindeki switch'e alışkanlıkla default eklemek. Bugün derlenir ve yarın eklemeyi unuttuğunuz her durumu gizler.

Eski ifade biçiminde break unutmak. Yeni kod yazıyorsanız ok biçimini kullanın ve sorun ortaya çıkamaz.

Bileşik atamanın yalnızca bir kısaltma olduğunu sanmak.

```
byte b = 10;
b += 300;      // compiles, prints 54
b = b + 300;   // does not compile
```

`+=` sessizce bir dönüşüm ekler. `10 + 300` sonucu `310`, ve bunu byte'a kesmek `54` verir. Açık biçim ise reddeder, ki ikisinden güvenli olanı budur.

Çalışma zamanında oluşturulan String'lerde `==` kullanmak.

```
a1 == a2 : true      // ikisi de derleme zamanı sabiti, havuzlanmış
a1 == a3 : false     // a3 bir StringBuilder ile oluşturuldu
```

Aynı metin, string'in ne zaman var olduğuna bağlı olarak farklı cevaplar.

Taşımaya dayanan bir döngü koşulu yazmak.

```
for (int i = 1; i > 0; i *= 2) { }
```

Bu gerçekten biter, ama yalnızca `i` değeri `Integer.MAX_VALUE` sınırını aşıp negatife sardığı için, ki bu yazdığınız sebep değil.

Özet Çıkarımlar

- `if` içine yalnızca `boolean` girer. Doğruluk kavramı ve yanlış değerler tablosu yok.
- `&&` ve `||` kısa devre yapar, bu da işlenen sırasını bir doğruluk meselesi haline getirir.
- **Switch ifadeleri değer üretir**, dal başına birkaç etiket alır ve asla alt duruma düşmez.
- `yield` bir switch bloğundan değer döndürür; `return` metottan çıkardı.
- **Kapalı kümede `default` yazmayın**, böylece durum eklemek sessizce düşmek yerine derlemeyi kırsın.
- **İndekse ihtiyacınız yoksa gelişmiş `for` kullanın.**
- `break` etiket; bayrak değişkeni olmadan iç içe döngülerden çıkar.

Ödev

[homework/README.md](#)

Switch ifadesiyle küçük bir not verme aracı yazın, sonra alt duruma düşme hatasını derleme hatasına çevirin. Referans çözüm [solutions/03-control-flow/](#) altında.